

Documentazione del Metodo TIMO

May 13, 2025

Contents

1	Introduzione a TIMO	2
2	Ruolo e Utilizzo dei Set di Dati	2
2.1	Training Set (Few-Shot)	2
2.2	Validation Set	3
2.3	Test Set	3
3	Estrazione e Caricamento delle Feature	4
3.1	Estrazione delle Feature (<code>extract_features_all.py</code>)	4
3.2	Caricamento delle Feature (<code>main.py</code>)	4
4	Funzionamento di TIMO	5
4.1	Image-Guided Text (IGT)	5
4.2	Classificazione e Valutazione	5
5	Esempio con il Dataset DTD (Describable Textures Dataset)	5
6	Considerazioni per il nostro lavoro	6
6.1	Artgraph_Artist	6
6.2	Artgraph_Style	6

1 Introduzione a TIMO

TIMO è un approccio sviluppato nell'ambito del *few-shot learning* che sfrutta modelli pre-addestrati come CLIP (Contrastive Language-Image Pre-training). L'obiettivo è migliorare la classificazione di immagini utilizzando un numero molto limitato di esempi di training per classe. TIMO opera principalmente estraendo e affinando feature visive e testuali, per poi utilizzarle in un processo di classificazione. Il codice fornito, in particolare `main.py` e `extract_features_all.py`, illustra il flusso di lavoro di TIMO e lo confronta con altri metodi *state-of-the-art* come Tip-Adapter, APE e GDA-CLIP.

La Figura 1 mostra uno schema generale del funzionamento di TIMO.

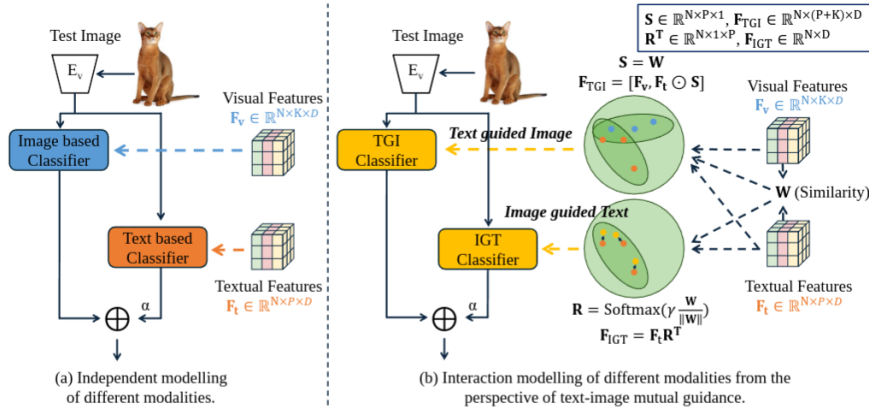


Figure 1: Schema generale del framework TIMO.

2 Ruolo e Utilizzo dei Set di Dati

TIMO, come molti approcci di machine learning, utilizza distinti set di dati per l'addestramento, la validazione e il test. La gestione di questi set è cruciale per il corretto funzionamento e la valutazione del metodo.

2.1 Training Set (Few-Shot)

Il training set in TIMO è tipicamente un set *few-shot*, il che significa che contiene solo un numero limitato di esempi per ogni classe (ad esempio, 1, 2, 4, 8, o 16 campioni, come specificato dall'argomento `--shot` e dalla variabile `k_shot` in `extract_features_all.py`).

Utilizzo nel codice:

1. **Costruzione del Cache Model:** Le immagini del training set vengono utilizzate per costruire un "cache model". Questo processo è gestito dalla funzione `extract_few_shot_feature` (e `extract_few_shot_feature_all`) in `extract_features_all.py`.
2. **Estrazione delle Feature Visive:** Per ogni immagine nel training set, vengono estratte le feature visive utilizzando l'encoder di immagini del modello CLIP (`clip_model.encode_image(images)`).
3. **Data Augmentation:** Per aumentare la robustezza e la generalizzazione con pochi campioni, viene applicata la data augmentation. La variabile `cfg['augment_epoch']` controlla il numero di volte in cui le feature vengono estratte con diverse augmentation. Le feature risultanti vengono mediate.
4. **Salvataggio delle Feature:** Le feature estratte (`cache_keys`) e le relative etichette in formato one-hot (`cache_values`) vengono salvate su disco (ad esempio, `keys_{shots}shots.pt` e `values_{shots}shots.pt`) nella directory specificata da `cfg['cache_dir']`.
5. **Caricamento in main.py:** Queste feature pre-calcolate vengono poi caricate in `main.py` tramite la funzione `load_few_shot_feature(cfg)`.

La Figura 2 mostra un esempio di immagine che potrebbe appartenere al training set del dataset DTD.

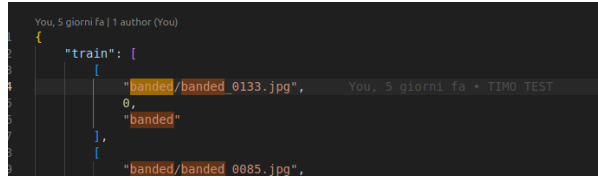


Figure 2: Esempio di immagine dal training set (ipoteticamente dal dataset DTD, ad es. una texture "interlacciata").

2.2 Validation Set

Il validation set è utilizzato per scopi intermedi, come l'ottimizzazione di iperparametri o la selezione di strategie, prima della valutazione finale sul test set.

Utilizzo nel codice:

1. **Estrazione delle Feature Visive:** Similmente al training set, le feature visive delle immagini del validation set vengono estratte usando `clip_model.encode_image(images)` e salvate su disco. Questo è gestito dalla funzione `extract_val_test_feature(cfg, "val", ...)` in `extract_features_all.py`. I file salvati sono tipicamente `val_f.pt` (features) e `val_l.pt` (labels).
2. **Caricamento in main.py:** Queste feature vengono caricate in `main.py` tramite `load_val_test_feature(cfg, "val")`.
3. **Ottimizzazione/Ricerca:** Nel caso di TIMO-S (TIMO con Search), il validation set (`val_features`, `val_labels`) è utilizzato dalla funzione `image_guide_text_search` per ottimizzare o cercare i parametri migliori per la guida delle feature testuali da parte delle feature immagine.

2.3 Test Set

Il test set è utilizzato esclusivamente per la valutazione finale delle prestazioni del modello. È fondamentale che questo set non venga utilizzato durante nessuna fase di training o ottimizzazione per garantire una stima imparziale della capacità di generalizzazione del modello.

Utilizzo nel codice:

1. **Estrazione delle Feature Visive:** Le feature visive delle immagini del test set vengono estratte e salvate in modo analogo al validation set, utilizzando la funzione `extract_val_test_feature(cfg, "test", ...)` in `extract_features_all.py`. I file salvati sono tipicamente `test_f.pt` e `test_l.pt`.
2. **Caricamento in main.py:** Le feature del test set vengono caricate tramite `load_val_test_feature(cfg, "test")` (o `"val"` per ImageNet).
3. **Valutazione Finale:** Le prestazioni di TIMO, TIMO-S e dei metodi baseline (Tip-Adapter, APE, GDA-CLIP) sono calcolate utilizzando le feature del test set (`test_features`, `test_labels`) e le rispettive strategie di classificazione. Le metriche (tipicamente l'accuracy) vengono poi registrate e salvate in `outputs/results.csv`.

La Figura 3 mostra un esempio di immagine che potrebbe appartenere al test set del dataset DTD.



Figure 3: Esempio di immagine dal test set (ipoteticamente dal dataset DTD, ad es. una texture "punteggiata").

3 Estrazione e Caricamento delle Feature

Un passaggio preliminare fondamentale nel flusso di lavoro di TIMO è l'estrazione delle feature, sia visive che testuali, che vengono poi salvate e caricate per l'elaborazione successiva. Questo approccio evita di dover rieseguire l'estrazione, computazionalmente onerosa, ad ogni esecuzione.

3.1 Estrazione delle Feature (`extract_features_all.py`)

Il file `extract_features_all.py` è responsabile della generazione e del salvataggio di tutte le feature necessarie.

- **Feature Visive:**

- `extract_few_shot_feature`: Estrae feature dalle immagini del training set (few-shot), applica augmentation e le salva come `cache_keys` e `cache_values`.
- `extract_val_test_feature`: Estrae feature dalle immagini dei set di validazione e test e le salva.

- **Feature Testuali:**

- `extract_text_feature` e `extract_text_feature_all`: Generano embedding testuali per ogni classe. Questo processo coinvolge:
 1. Nomi delle classi (es. dal dataset).
 2. Template di prompt (es. "una foto di una").
 3. Eventuali prompt aggiuntivi da file JSON (es. prompt generati da GPT o specifici del dataset come CuPL, specificati in `dataset.cupl_path`).

Questi testi vengono tokenizzati e passati all'encoder testuale di CLIP (`clip_model.encode_text`) per ottenere gli embedding, che vengono poi normalizzati e salvati (es. `text_weights_cupl_t_all.pt`).

La Figura 4 illustra schematicamente il processo di estrazione delle feature.

```
219 train_loader_normalizer = Normalizer(mean=[0.48145866, 0.4578275, 0.40821073], std=[0.20102934, 0.20102934, 0.20102934])
220 train_loader_cache = build_data_loader(data_source=dataset.train_x, batch_size=256, tfm=train_transform, is_train=True, shuffle=True)
221
222 # Construct the cache model by few-shot training set
223 print("\nConstructing cache model by few-shot visual features and labels.")
224 extract_few_shot_feature(cfg, clip_model, train_loader_cache)  # 700, 5 giorni fa + TIMO TEST
225 extract_few_shot_feature_all(cfg, clip_model, train_loader_cache, norm=norm)
226
227
228 # Extract val/test features
229 print("\nLoading visual features and labels from val and test set.")
230 extract_val_test_feature(cfg, "val", clip_model, val_loader, norm=norm)
231 if not set == 'imagenet':
232     extract_val_test_feature(cfg, "test", clip_model, test_loader, norm=norm)
233
234 # [dataset.cupl_path, dataset.woffle_path, dataset.CLIP path]
235 extract_text_feature(cfg, dataset.classnames, [dataset.cupl_path], clip_model, dataset.template, use_gpt_prompt=False)
236 extract_text_feature(cfg, dataset.classnames, [dataset.cupl_path], clip_model, dataset.template)
237 extract_text_feature_all(cfg, dataset.classnames, [dataset.cupl_path], clip_model, dataset.template, norm=norm)
```

Figure 4: Processo di estrazione delle feature visive e testuali utilizzando CLIP.

3.2 Caricamento delle Feature (`main.py`)

Il file `main.py` inizia caricando le feature pre-estratte e salvate nella directory `cache_dir`.

- **Feature Testuali:** Caricate da file `.pt`, ad esempio `[breakable]clip_weights_cupl_all = torch.load(cfg['cache_dir'] + "/text_weights_cupl_t_all.pt")`. Queste possono essere ulteriormente processate (es. media sui prompt) per ottenere i pesi del classificatore testuale di CLIP.
- **Feature del Training Set (Cache Model):** Caricate tramite `[breakable]cache_keys, cache_values = load_few_shot_feature(cfg)`.
- **Feature dei Set di Validazione e Test:** Caricate tramite `[breakable]val_features, val_labels = load_val_test_feature(cfg, "val")` e `test_features, test_labels = load_val_test_feature(cfg, "test")`.

La Figura 5 illustra il caricamento delle feature pre-calcolate.

```

clip_weights_cupl = clip_weights_cupl.all.mean(dim=1).t()
clip_weights_cupl = clip_weights_cupl / clip_weights_cupl.norm(dim=0, keepdim=True)

You, 5 giorni fa • TIMO TEST -

# Construct the cache model by few-shot training set
print("\nConstructing cache model by few-shot visual features and labels.")
cache_keys, cache_values = load_few_shot_feature(cfg)

# Pre-load val features
print("\nLoading visual features and labels from val set.")
val_features, val_labels = loda_val_test_feature(cfg, "val")

# Pre-load test features
print("\nLoading visual features and labels from test set.")
if cfg['dataset'] == 'imagenet':
    test_features, test_labels = loda_val_test_feature(cfg, "val")
else:
    test_features, test_labels = loda_val_test_feature(cfg, "test")

```

Figure 5: Caricamento delle feature pre-estratte in `main.py`.

4 Funzionamento di TIMO

Una volta caricate tutte le feature necessarie, TIMO procede con il suo meccanismo di fusione e classificazione.

4.1 Image-Guided Text (IGT)

Un aspetto centrale di TIMO sembra essere la capacità di utilizzare le poche feature visive del training set per "guidare" o raffinare le feature testuali.

1. **Creazione di "Image Weights"**: Dalle `cache_keys` (feature visive del training set), vengono calcolati dei pesi medi per immagine per ogni classe (`image_weights`). Questi rappresentano una sorta di prototipo visivo per ciascuna classe, basato sui pochi esempi disponibili.
2. **Guida delle Feature Testuali**: La funzione `image_guide_text` (e la sua variante con ricerca `image_guide_text_search` usata in TIMO-S) prende in input le feature testuali originali (es. `clip_weights_cupl_all`) e i pesi delle immagini (`image_weights`) per produrre delle feature testuali raffinate (`clip_weights_IGT`). Viene calcolato anche un `matching_score`.

4.2 Classificazione e Valutazione

Con le feature testuali raffinate (`clip_weights_IGT`) e, in alcuni casi, le feature testuali originali e il `matching_score`, TIMO esegue la classificazione sulle feature del test set.

- **TIMO**: Utilizza `clip_weights_IGT`, `clip_weights_cupl_all` e `matching_score` per la classificazione. La funzione `TIMO(...)` in `main.py` implementa questa logica.
- **TIMO-S**: Simile a TIMO, ma `clip_weights_IGT` e `matching_score` sono ottenuti tramite una fase di ricerca (`image_guide_text_search`) che utilizza il validation set per ottimizzare i parametri della guida.

Le prestazioni (accuracy) vengono calcolate confrontando le predizioni con le `test_labels`. I risultati vengono poi salvati, ad esempio in un file CSV (`outputs/results.csv`).

5 Esempio con il Dataset DTD (Describable Textures Dataset)

Il Describable Textures Dataset (DTD) è un dataset di immagini di texture, ed è uno dei dataset supportati da TIMO (come indicato in `datasets/__init__.py` e utilizzato in `extract_features_all.py`).

Il flusso di lavoro con DTD sarebbe il seguente:

1. Setup del Dataset:

- Le immagini si trovano in `$DATA/dtd/images/`.
- Le etichette e i nomi delle classi sono in `$DATA/dtd/labels/`.
- La suddivisione in train, validation e test è definita nel file JSON `$DATA/dtd/split_zhou_DescribableTextures`. Il caricamento e la gestione di queste suddivisioni sono gestiti dalla classe `DescribableTextures` in `datasets/dtd.py` (non fornito, ma deducibile da `datasets/__init__.py`) e dalle utility in `datasets/utlis.py`.

2. Estrazione Feature (come da Sezione 3 e 4.1):

- **Training (Few-Shot):** Un piccolo numero di immagini per ogni classe di texture (es. "banded", "blotchy", "braided", come visibile in `TrainEx.png` che potrebbe rappresentare una texture "interlacciata" o "a rete") viene selezionato dal set di training di DTD. Le loro feature visive vengono estratte e salvate come `cache_keys` e `cache_values`.
- **Validation:** Le immagini dal set di validazione di DTD vengono processate per estrarre `val_features` e `val_labels`.
- **Test:** Le immagini dal set di test di DTD (es. una texture "punteggiata" come in `TestEx.png`) vengono processate per estrarre `test_features` e `test_labels`.
- **Testuali:** Le feature testuali per le classi di DTD (es. "banded", "blotchy", etc.) vengono estratte usando i template e i prompt specifici per DTD.

3. Esecuzione di TIMO (come da Sezione 4.2 e 5):

- Le feature estratte vengono caricate.
- TIMO utilizza le feature del training set di DTD per raffinare le feature testuali delle texture.
- Il modello viene valutato sulle feature del test set di DTD per misurare la sua capacità di classificare correttamente le texture mai viste prima, basandosi su pochi esempi.

Questo processo permette a TIMO di adattarsi e classificare efficacemente le diverse texture presenti nel dataset DTD, anche con un supporto di training limitato. I risultati, inclusa l'accuracy media per il dataset DTD con un certo numero di shot e seed, vengono salvati nel file `outputs/results.csv`.

Come possibile vedere dalle immagini `TrainEx.png` e `TestEx.png`, gli autori di TIMO hanno inserito le stesse classi sia nel training set che nel test set (ovviamente non inserendo le stesse immagini).

6 Considerazioni per il nostro lavoro

Dato che il nostro lavoro si basa su TIMO, è fondamentale comprendere a fondo il suo funzionamento e replicarlo. Il nostro task prevedere di effettuare FSL con $k=[1,2,4,8,16]$

6.1 Artgraph_Artist

In questo caso le classi sono gli artisti e le immagini sono le loro opere. Dobbiamo organizzare il dataset affinché sia compatibile con TIMO e con il task di few-shot learning. Per questo motivo viene proposta la seguente divisione in training, validation e test set:

- **Test set:** Dato che dobbiamo eseguire task fino a 16-shot, il test set deve contenere almeno 16 immagini per ogni artista. Per questo motivo, il test set è composto da 16 immagini per ogni artista. Le immagini sono state scelte in modo casuale tra le opere di ogni artista.
- **Training e validation set:** Per il training e il validation set si potrebbero inserire rispettivamente 10 e 5 immagini per artista.
- **Artisti con poche opere:** Gli artisti che non presentano opere sufficienti per creare i set verranno esclusi dal dataset.

6.2 Artgraph_Style

In questo caso le classi sono gli stili e le immagini sono le opere. Dobbiamo organizzare il dataset affinché sia compatibile con TIMO e con il task di few-shot learning. Prendiamo le 10 classi più rappresentate e creiamo i set di training, validation e test come segue:

- **Test set:** Dato che dobbiamo eseguire task fino a 16-shot, il test set deve contenere almeno 16 immagini per ogni stile. Un buon approccio potrebbe essere quello di prendere tutti gli artisti che partecipano a quello stile e prendere 5 immagini di quell'artista in quello stile. (senza alcun problema si dovrebbero raggiungere le 16 immagini per stile)
- **Training e validation set:** Per il training e il validation set si potrebbero inserire rispettivamente 20 e 10 immagini per stile randomicamente

- **Stili con poche opere:** Gli stili che non presentano opere sufficienti per creare i set verranno esclusi dal dataset.